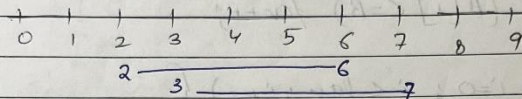


Class 13: Array 3: Interview Problems

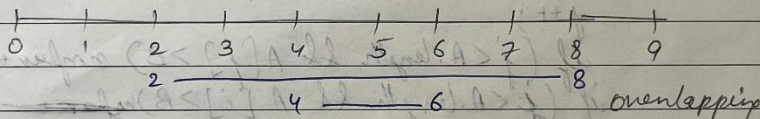
② Merge Intervals

- ① I_1 I_2 let's say we have 2 intervals, 2, 4 I_2
 $(2, 6)$ $(3, 7)$ with coordinates,

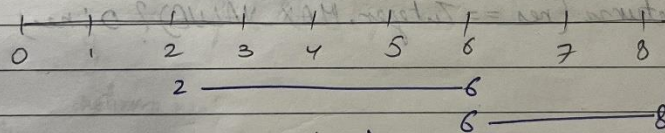


Interval is defined as start & end point where $start_1 \leq end_1$
 This is overlapping.

- ② $(2, 6)$ $(4, 6)$

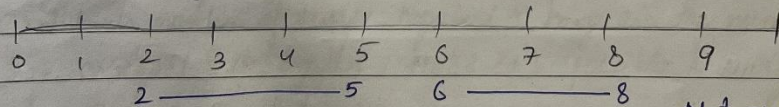


- ③ $(2, 6)$ $(6, 8)$

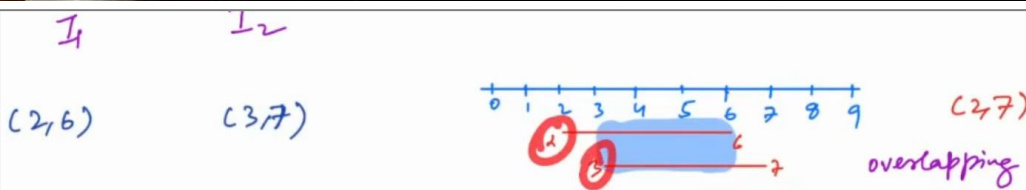


overlapping if
 they are touching,

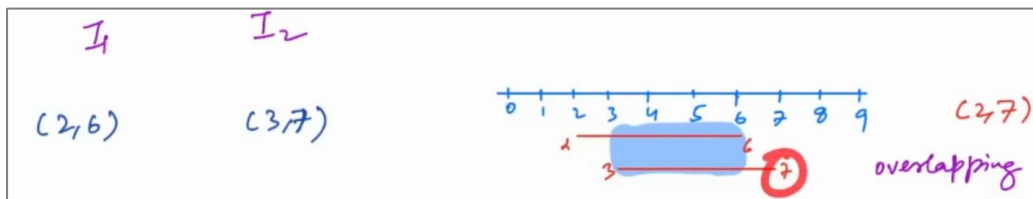
- ④ $(2, 5)$ $(6, 8)$



Not overlapping.



Whichever is the smaller value of the start will be the starting of the final interval.
Similarly whichever is the larger end will be the ending of the final interval.



Non-overlapping condition

$I_1 = (s_1, e_1)$
 $I_2 = (s_2, e_2)$

Conditions for non-overlapping intervals:

- $e_1 < s_2$ (circled in red)
- $e_2 < s_1$ (circled in red)

Merge 2 overlapping intervals

For $(2, 6)$ & $(3, 7) \rightarrow$ merge will be $(2, 7)$.

Start = $\min(s_1, s_2)$
 End = $\max(e_1, e_2)$

Q1 Given a list of intervals, sorted wr.t. start time. Merge all overlapping interval & return the sorted list.

Start = $[0, 2, 5, 6, 7, 8, 12]$

End = $[2, 4, 6, 8, 10, 9, 14]$

Intervals: $0-2, 5-6, 7-10, 1-9, 6-8, 8-9, 12-14$

Merged intervals: $0-4, 5-10, 12-14$

O/p: $(0, 4), (5, 10), (12, 14)$

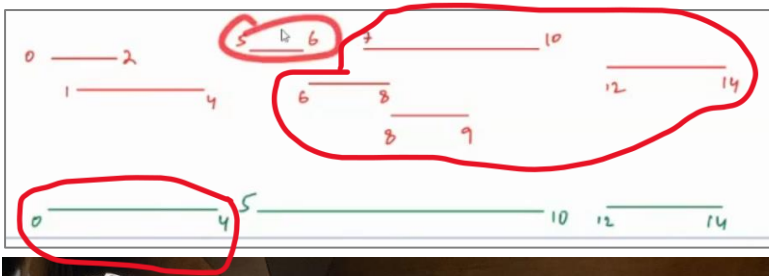
Condition for overlapping:

① $s_i \leq s_{i+1}$ (always true as sorted)

② $s_i \leq e_i$ (always true)

③ $s_{i+1} \leq e_i$ (This is overlapping condition)

Diagram showing the condition $s_{i+1} \leq e_i$ for overlapping intervals.



Since 5 & 6 is not overlapping with 0 to 4 so 0 to 4 will not overlap with any other intervals. Hence we can print 0 to 4 intervals as one of our answer

Code

```

S = st[0], E = end[0]
for (i=1 to n-1) {
    if (st[i] <= E) {
        E = max(E, end[i])
    }
    else {
        // not overlapping
        print(S, E)
        S = st[i], E = end[i]
    }
}
print(S, E) // last interval

```

Updating Interval condition

$\min(S, st[i]) \rightarrow$ not needed
 $\max(E, end[i])$ // we have to always be less than $st[i]$

if overlapping $\rightarrow$ merge the end intervals

TC: $O(N)$
SC: $O(1)$

Dry run:

st = [0 1 5 6 7 8 12] S = 0 (5) 12
 8 < 10
 end = [2 4 6 8 10 9 14] E = 2 4 6 8 10 14

Q1 [Scaler wants to do maintenance. Find the longest period of no user activity.] Let's say Scaler is planning to conduct maintenance on its website and to conduct this maintenance, they have to bring down the website for sometime. Now they want to schedule the maintenance such that there is no user activity in that duration. You are given an array of hours of all the learners, let's say learner 1 is active from 9 to 11 in a day & so on. Find the longest period where no learner is active & in that slot you want to do the maintenance.

DATA = [(9, 11), (14, 16), (15, 20)]

$\downarrow$ after merging

[(9, 11), (14, 20)]

$\rightarrow$ after 11 to 14 $\rightarrow$ no user activity $\rightarrow$ 3 hrs

marginal overlapping interval

$$TC = O(N \times ans)$$

If ans = 100, then the loop would have run 100 times. If ans = 4 then the loop would have run 4 times. and N = length of the array. What is the max value ans can have? $\rightarrow N+1$ coz if all the elements are present $A = [1, 2, 3, 4, 5]$ and our ans would be 6. so $N+1$

0hr. $[(9, 11) \xrightarrow{3\text{ hours}} (14, 20)] \xrightarrow{2\text{ hours}} (16, 18)$
 9hr. 3hr. 4hr.

// all merging all the overlapping intervals, you will get chunks whenever user is active

2nd Use case :- Calendar when you have a schedule meeting & you merge all of their non-overlapping interval where no person has a meeting.

Q3 Given an unsorted array of integers. Find 1st missing natural #s. $\#s. \geq 1$

✓ $A = [3, -2, 1, 7, 2]$

Ans = 4 $\rightarrow$ 1 is present, 2 is present, 3 is present but 4 is missing

$A = [4, 1, 3, 2]$

Ans = 5

① Brute force :-

Start from 1 and check which no. is not present in array

$$TC = O(N \times ans)$$

$\rightarrow$ If ans = 4, then you have run loop 4 times

Over For 1, you will iterate over array $O(N)$ times

For 2, " " " " " " $O(N) \times 4$ times

max. value of ans? = $N+1$.

worst case is where $A = [1, 2, 3, \dots, N]$ in array, & 1st missing #

would be $N+1$.

$$TC = O(N^2)$$

$$SC = O(1)$$

min ans, possible = 1

max " " = $N+1$

$$1 \leq ans \leq N+1$$

② Hash set

③
we create an array of size = 8 to mark True

$A = [5, 3, 1, -1, -2, -4, 7, 2]$ $N = 8$

→

T	T	T		T		T	
1	2	3	4	5	6	7	8

4 is the 1st missing index, we can say this is the ans.

$TC = O(N)$

$SC = O(N)$ → creating an array takes space as $O(N)$

④ let's assume, all $a[i]$ are +ve. we can use the i/p array to mark True & False.

Optimized Approach

$A = \begin{matrix} 0 & 1 & 2 & 3 & 4 \\ [3 & 10 & 1 & 7 & 2] \end{matrix}$

if $a[i]$ is -ve $\Rightarrow (i+1)$ is present in array

Code

for ($i = 0$ to $n-1$) {

$x = \text{abs}(a[i])$

// ignore -ve sign if $a[i] = -3$ it will give 3

if ($x \geq 1$ & $x \leq N$) {

$idx = x - 1$

if ($a[idx] > 0$)

$a[idx] \neq -1$ // change the value to -1.

}

for ($i = 0$ to $n-1$) {

if ($a[i] > 0$)

return $i+1$ &

}

return $n+1$.

// false condition

1st missing natural #
✓

Prq run

$A = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ -3 & & -1 \end{bmatrix}$

$i = 1$

$x = 4$

$x \geq 1 \& \& x \leq N \times$

$idx = 4 - 1 = 3$ Not in range.

$i = 2$

$x = 1$

$x \geq 1 \& \& x \leq N \checkmark$

$idx = 1 - 1 = 0$

if $(a[0] > 0) \checkmark$

$a[0] \neq -1$

?

y

for $(i = 0 \text{ to } n-1)$ {

if $(a[0] > 0) \times$

if $a[1] > 0 \checkmark$

return $1 + 1 = 2$

ans = 2

$A = \begin{bmatrix} 0 & 1 & 2 \\ 10 & 12 & 13 \end{bmatrix}$

$N = 3$ ans = 1 since None of the elements are in the range 0-3.

$A = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 2 \\ -1 & -2 & \end{bmatrix}$ ans = 3

$A = \begin{bmatrix} -1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix}$ ans = $2 + 1 = 3$

2. How to resolve for negative numbers?
will -ve # even be our answer?

NO $\rightarrow$ so we don't care about that. So should we make them +ve? No $\rightarrow$ we can't if we ignore their negative sign, they will fall in 0 to N range.

INTEGER. MAX-VALUE $\sim 10^9$

Page No.:
Date: YOUVA

Should we make them 0? If we mark them as zero, how we will mark them as -ve. NO, Change them to any big +ve no. $\rightarrow N+2$ coz $N+1$ cannot be in our answer.

Code

```
for (i=0 to n-1) {
    if (a[i] <= 0) { // we have to include zero also
        a[i] = N+2
    }
    for (i=0 to n-1) {
        x = abs(a[i])
        if (x >= 1 && x <= N) {
            idx = x-1
            if (a[idx] > 0)
                a[idx] = -1
        }
    }
    for (i=0 to n-1) {
        if (a[i] > 0)
            return i+1
    }
    return n+1 // false condition
}
```

1st missing
natural #
with -ve sign

$A = [2, -10, 1]$

Assignment
Q1.

First Missing Integer: Given an unsorted integer array, A of size N. Find the 1st missing +ve integer. Note: Your algorithm should run in $O(n)$ time and use constant space. O/P: Return an integer denoting the first missing +ve integer.

i/p: $[3, 4, -1, 1]$

O/p: 2

Explanation: First +ve integer missing from the array is 2.

Problem Statement -1 - First Missing Integer

Problem Description

Given an unsorted integer array, **A** of size **N**. Find the first missing positive integer.

Note: Your algorithm should run in $O(n)$ time and use constant space.

Problem Constraints

$1 \leq N \leq 1000000$

$-10^9 \leq A[i] \leq 10^9$

Input Format

First argument is an integer array A.

Output Format

Return an integer denoting the first missing positive integer.

Example Input

Input 1: [1, 2, 0]

Input 2: [3, 4, -1, 1]

Input 3: [-8, -7, -6]

Example Output

Output 1: 3

Output 2: 2

Output 3: 1

Example Explanation

Explanation 1:

A = [1, 2, 0]

First positive integer missing from the array is 3.

Explanation 2:

A = [3, 4, -1, 1]

First positive integer missing from the array is 2.

Explanation 3:

A = [-8, -7, -6]

First positive integer missing from the array is 1.

Actual Code

```
public class Solution {
    public int firstMissingPositive(int[] A) {
        int n = A.length;
        int x = 0, idx = 0;
        for(int i = 0; i < n; i++){
            if(A[i] <= 0){
                A[i] = n+2;
            }
        }
        for(int i = 0; i < n; i++){
            x = Math.abs(A[i]);
```



```

        if(x >=1 && x<=n){
            idx = x-1;
            if(A[idx] > 0){
                A[idx] *= -1;
            }
        }
    }
    for(int i=0; i<n; i++){
        if(A[i] >0){
            return i+1;
        }
    }
    return n+1;
}
}

```

Problem Statement – 2 - Merge Sorted Overlapping Intervals – 2

Problem Description

You are given a collection of intervals **A** in a **2-D array format**, where each interval is represented by a pair of integers `[start, end]`. The intervals are sorted based on their start values.

Your task is to **merge** all overlapping intervals and return the resulting set of **non-overlapping intervals**.

Problem Constraints

1 <= len(A) <= 100000.

1 <= A[i][0] <= A[i][1] <= 100000

A is sorted based on the start value (A[i][0])

Input Format

First argument is a list of intervals in 2-Dimensional Array.

Output Format

Return the sorted list of intervals after merging all the overlapping intervals.

Example Input

Input 1:

[[1, 3], [2, 6], [8, 10], [15, 18]]

Input 2:

[[2, 10], [4, 9], [6, 7]]

Example Output

Output 1:

[[1, 6], [8, 10], [15, 18]]

Output 2:

[[2, 10]]

Example Explanation

Explanation 1:

Merge intervals [1,3] and [2,6] -> [1,6].

so, the required answer after merging is [1,6],[8,10],[15,18].

No more overlapping intervals present.

Explanation 2:

Merge intervals [2, 10], [4, 9], [6, 7] as [2,10].

Since [4, 9] and [6, 7] is overlapping inside the interval [2, 10].

so, the required answer after merging is [2, 10].

Actual Code

```
import java.util.ArrayList;
import java.util.List;

public class Solution {
    public int[][] solve(int[][] A) {
        List<int[]> merged = new ArrayList<>();

        int s = A[0][0];
        int e = A[0][1];

        for(int i = 1; i<A.length; i++){
            if(A[i][0] <= e){
                e = Math.max(e, A[i][1]);
            }
            else{
                merged.add(new int[]{s,e});
                s = A[i][0];
                e = A[i][1];
            }
        }
        merged.add(new int[]{s,e});
        return merged.toArray(new int[merged.size()][]);
    }
}
```

Problem Statement – 3 - Merge Intervals – 2

Problem Description

You have a set of non-overlapping intervals. You are given a new interval [start, end], insert this new interval into the set of intervals (merge if necessary).

You may assume that the intervals were initially sorted according to their start times.

Problem Constraints

$0 \leq |\text{intervals}| \leq 10^5$

Input Format

First argument is the vector of intervals

second argument is the new interval to be merged

Output Format

Return the vector of intervals after merging

Example Input

Input 1:

Given intervals [1, 3], [6, 9] insert and merge [2, 5] .

Input 2:

Given intervals [1, 3], [6, 9] insert and merge [2, 6] .

Example Output

Output 1:

[[1, 5], [6, 9]]

Output 2:

[[1, 9]]

Example Explanation

Explanation 1:

(2,5) does not completely merge the given intervals

Explanation 2:

(2,6) completely merges the given intervals

Actual Code

```
import java.util.ArrayList;
import java.util.List;
public class Solution {
    public int[][] insert(int[][] A, int[] B) {
        List<int[]> merged = new ArrayList<>();
        int i = 0, n = A.length;
        while(i < n && A[i][1] < B[0]){
            merged.add(A[i]);
            i++;
        }

        int start = B[0], end = B[1];
        while(i < n && A[i][0] <= end){
            start = Math.min(start, A[i][0]);
            end = Math.max(end, A[i][1]);
            i++;
        }
        merged.add(new int []{start,end});
        while(i < n){
            merged.add(A[i]);
            i++;
        }
        return merged.toArray(new int[merged.size()][]);
    }
}
```

Once B and overlapping intervals are merged, add all remaining intervals from A.

